

Semantic Video Retrieval System

Project Roadmap | Natural-Language Video Search Powered by CLIP + Qdrant

This roadmap outlines the technical plan for building a semantic video search system that lets users find relevant video moments using natural-language queries. The system samples keyframes from video, encodes them with a vision-language model (CLIP), stores the resulting vectors in a Qdrant vector database, and retrieves the closest matching frames for a given text query through an interactive Streamlit interface.

System Architecture & Workflow

```
[ Input Video ] --> [ Keyframe Sampler (OpenCV) ] --> [ Vision Encoder (CLIP) ] --> [ Store  
in Qdrant Vector DB ]  
|  
[ User Search ] --> [ Natural Language Query ] --> [ Text Encoder (CLIP) ] -----  
Similarity Search  
|  
[ Ranked Results Display ] <-- [ Streamlit UI Video Player ] <-----
```

The pipeline has two parallel tracks that converge at similarity search: an **ingestion track** (video → keyframes → embeddings → Qdrant) and a **query track** (text → embedding → nearest-neighbor search), with results surfaced through a video-player-enabled dashboard.

Work Package Roadmap

WP1 Environment Setup & Core Benchmarking

- **Local GPU Setup** — Configure PyTorch with CUDA acceleration and verify all model-execution dependencies.
- **Vector DB Initialization** — Stand up a local or Docker-based Qdrant instance to store and search frame vector embeddings.
- **Model Benchmarking** — Compare OpenAI CLIP (ViT-B/32) against ViCLIP on sample clips for extraction speed, memory footprint, and semantic accuracy.
- **Sandbox Data Prep** — Download and organize a small 50–100 video test corpus for local pipeline validation.

WP2 Video Processing & Feature Extraction Engine

- **Frame Extraction Pipeline** — Build an OpenCV/PyAV module to extract keyframes via scene transitions or fixed intervals (e.g., 1 fps).
- **Metadata Payload Tagging** — Pair each frame with video ID, source path, frame index, and start/end timestamps.
- **Batch Embedding Generation** — Run keyframes through the vision encoder in mini-batches to control memory usage.

WP3**Vector Database Schema & Indexing**

- **Qdrant Collection Creation** — Initialize collections using cosine similarity, matching the embedding dimension (512-D for CLIP ViT-B/32).
- **Hierarchical Indexing** — Video-level averaged embeddings for coarse filtering, plus frame-level embeddings mapped to timestamps for precise retrieval.
- **Ingestion Pipeline Execution** — Batch-upload frame embeddings and metadata payloads to Qdrant.

WP4**Retrieval Engine & Interactive Interface**

- **Query Processing Engine** — Convert natural-language queries (e.g., "person opening a door at night") into text embeddings via the CLIP text encoder.
- **Similarity Search Logic** — Run Approximate Nearest Neighbor (ANN) queries in Qdrant, ranked by similarity score.
- **Streamlit Dashboard** — Text search UI plus result cards with similarity scores, thumbnails, and a video player that jumps to the matched segment.

WP5**Final Polish, Documentation & Compliance**

- **Compliance Checks** — Complete and sign the required administrative undertaking document.
- **System Documentation** — Architecture diagrams, database schema definitions, and performance benchmarks for final presentation.

Recommended Dataset Strategy

- **For Pipeline Testing** — Use the MSR-VTT test split via Hugging Face. Sets up in under 10 minutes and provides ready-made text–video ground truth.
- **For Live Presentation** — Index a custom subset of 30–50 high-resolution Pexels clips for crisp visuals and fast search speed during evaluation.

Roadmap generated from project task-breakdown documentation.